LR(1)

LR(1) parsing table

State	ACTION[INT]	ACTION[+]	ACTION[*]	ACTION[(]	ACTION[)]	ACTION[$]	GOTO[E]	GOTO[T]	GOTO[F]
0	s1			s2			3	4	5
1		r4 (F -> INT)	r4 (F -> INT)			r4 (F -> INT)			
2	s6			s7			8	9	10
3		s11				acc			
4		r1 (E -> T)	s12			r1 (E -> T)			
5		r3 (T -> F)	r3 (T -> F)			r3 (T -> F)			
6		r4 (F -> INT)	r4 (F -> INT)		r4 (F -> INT)				
7	s6			s7			13	9	10
8		s14			s15				
9		r1 (E -> T)	s16		r1 (E -> T)				
10		r3 (T -> F)	r3 (T -> F)		r3 (T -> F)				
11	s1			s2				17	5
12	s1			s2					18
13		s14			s19				
14	s6			s7				20	10
15		r5 (F -> (E))	r5 (F -> (E))			r5 (F -> (E))			
16	s6			s7					21
17		r0 (E -> E + T)	s12			r0 (E -> E + T)			
18		r2 (T -> T * F)	r2 (T -> T * F)			r2 (T -> T * F)			
19		r5 (F -> (E))	r5 (F -> (E))		r5 (F -> (E))				
20		r0 (E -> E + T)	s16		r0 (E -> E + T)				
21		r2 (T -> T * F)	r2 (T -> T * F)		r2 (T -> T * F)				

Активация Windows

We construct LR(1) table by calculating the closure of the initial state. Starting from the closure, we calculate all other states using the production rules. All states have the current production rules and their lookaheads (nonterminal that can follow the current production rule). That is, for production rule $[A \to \alpha \cdot B\beta,\ a]$ a is its lookahead. To calculate the closure, for every production rule $B \to \gamma$, we have to add $\boxed{[B \to \cdot\gamma,\ b]}$ in the closure, for every b $\boxed{b \in FIRST(\beta a)}$.

To construct all state we use GOTO(I, a) – the state that we get if we move the dot in the current state I through symbol a.

$GOTO(I, X) = CLOSURE(J)$, where

$J = \{[A \to \alpha X \cdot \beta, a] \mid [A \to \alpha \cdot X\beta, a] \in I\}$

We repeat this process until all states are added.

After constructing all states, we now have to construct the table. For a production rule $[A \to \alpha \cdot a\beta, b] \in I_i$ in state I we must add 'shift J' when we encounter terminal a, if $GOTO(I_i, a) = I_j$. If we are finishing our production rule, we must reduce but only when we encounter the lookahead. $[A \to \alpha\cdot, a] \in I_i$ then $ACTION[i, a] = reduce\ (A \to \alpha)$. Lastly, we need to accept if we are completing the starting nonterminal $[S' \to S\cdot, \$] \in I_i$. $ACTION[i, \$] = accept$.

SLR(1)

SLR(1) parsing table

State	ACTION[INT]	ACTION[+]	ACTION[*]	ACTION[(]	ACTION[)]	ACTION[$]	GOTO[E]	GOTO[T]	GOTO[F]
0	s1			s2			3	4	5
1		r4 (F -> INT)	r4 (F -> INT)		r4 (F -> INT)	r4 (F -> INT)			
2	s1			s2			6	4	5
3		s7				acc			
4		r1 (E -> T)	s8		r1 (E -> T)	r1 (E -> T)			
5		r3 (T -> F)	r3 (T -> F)		r3 (T -> F)	r3 (T -> F)			
6		s7			s9				
7	s1			s2				10	5
8	s1			s2					11
9		r5 (F -> (E))	r5 (F -> (E))		r5 (F -> (E))	r5 (F -> (E))			
10		r0 (E -> E + T)	s8		r0 (E -> E + T)	r0 (E -> E + T)			
11		r2 (T -> T * F)	r2 (T -> T * F)		r2 (T -> T * F)	r2 (T -> T * F)			

Активация Windows

SLR(1) uses the LR(0) states (meaning we have no lookaheads in the states) and difference is that we can reduce with any terminal in FOLLOW set. For example, for finished production $A \to \alpha\cdot$ we can reduce with any terminal in FOLLOW(A).

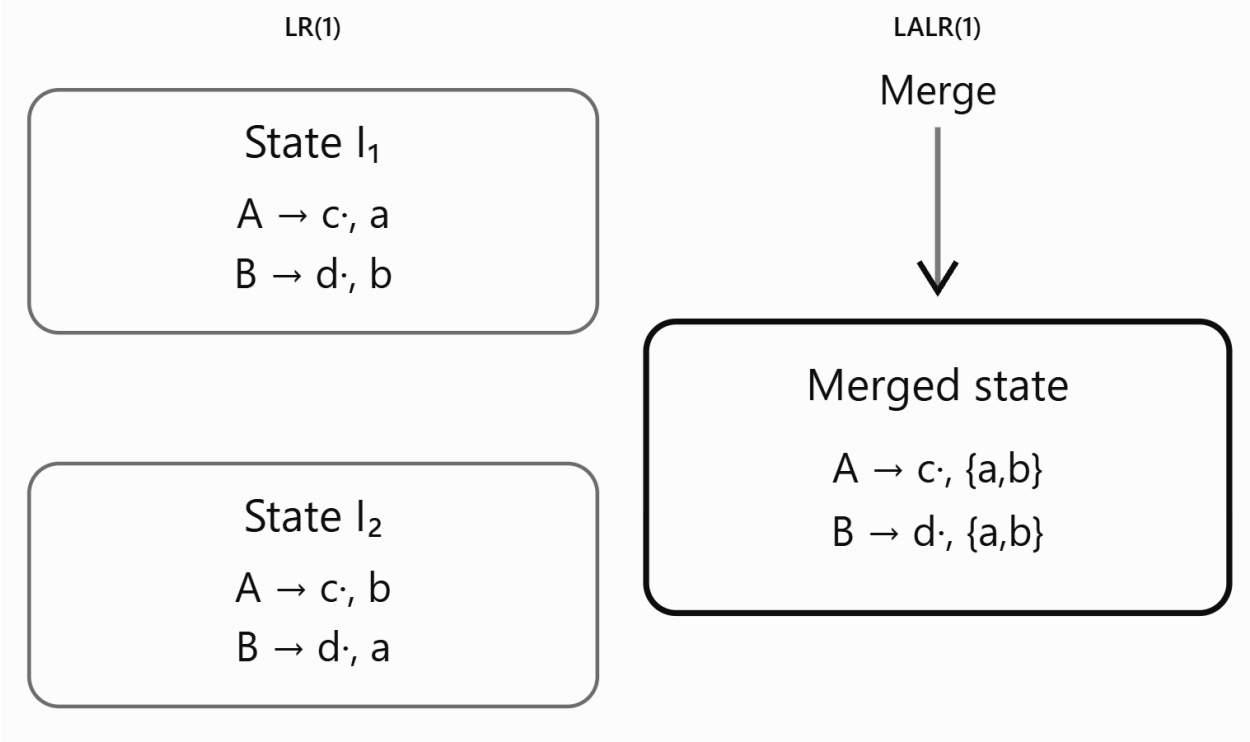
LALR(1)

LALR(1) parsing table

State	ACTION[INT]	ACTION[+]	ACTION[*]	ACTION[(]	ACTION[)]	ACTION[$]	GOTO[E]	GOTO[T]	GOTO[F]
0	s1			s2			3	4	5
1		r4 (F -> INT)	r4 (F -> INT)		r4 (F -> INT)	r4 (F -> INT)			
2	s1			s2			6	4	5
3		s7				acc			
4		r1 (E -> T)	s8		r1 (E -> T)	r1 (E -> T)			
5		r3 (T -> F)	r3 (T -> F)		r3 (T -> F)	r3 (T -> F)			
6		s7			s9				
7	s1			s2				10	5
8	s1			s2					11
9		r5 (F -> (E))	r5 (F -> (E))		r5 (F -> (E))	r5 (F -> (E))			
10		r0 (E -> E + T)	s8		r0 (E -> E + T)	r0 (E -> E + T)			
11		r2 (T -> T * F)	r2 (T -> T * F)		r2 (T -> T * F)	r2 (T -> T * F)			

Активация Windows

LALR(1) construction is mostly similar with LR(1) with one difference – we merge all states that have the same LR(0) core and combine their lookahead sets.


LR(1)
LALR(1)
Merge
State I_1
A → c·, a
B → d·, b
State I_2
A → c·, b
B → d·, a
Merged state
A → c·, {a,b}
B → d·, {a,b}